

Introduction to Lean

Programming, Propositions as Types, and Formal Proof

Jasper Robb, University of Winnipeg

Edited and typeset from the June 24 seminar transcript

Abstract

These notes reorganize the full seminar presentation into a coherent introduction to Lean. The central theme is that Lean is simultaneously a programming language and a proof assistant: its proof-assistant behavior emerges from its type theory. Starting from inductive types and pattern matching, the notes develop Boolean operations, the unit and empty types, natural numbers, products and sums, the propositions-as-types correspondence, constructive and classical logic, equality, and a recursive proof of associativity of addition. The final section records the seminar discussion on trust, formalization practice, learning resources, and editor support.

Contents

1	Opening and Overview	3
2	Lean as a Programming Language	3
2.1	Namespaces and inductive types	3
2.2	Booleans	3
2.2.1	Negation by pattern matching	4
2.2.2	Disjunction, NAND, and conjunction	4
2.3	The unit and empty types	5
2.3.1	The unit type	5
2.3.2	The empty type	5
2.3.3	Functions between Unit and Empty	5
2.4	Natural numbers	6
2.4.1	Addition	6
2.5	Products and sums	7
2.5.1	Product types	7
2.5.2	Sum types	7
3	Lean as a Proof Assistant	8
3.1	Logical connectives as type constructors	8
3.2	Reintroducing the logical names	8
4	Elementary Proofs by Construction and Pattern Matching	9
4.1	Projection from a conjunction	9

4.2	The first De Morgan implication	9
4.3	The converse De Morgan implication	10
5	Constructive Logic and Classical Logic	10
5.1	Law of excluded middle	11
5.2	Classical proof of the remaining De Morgan implication	11
6	Equality as an Inductive Family	11
6.1	Definition	11
6.2	Definitional equality and computation	12
6.3	Symmetry	12
6.4	Transitivity	12
6.5	Congruence under functions	13
7	Associativity of Natural-Number Addition	13
7.1	Base case	13
7.2	Successor case	13
8	How Lean Is Used in Practice	14
8.1	Graph theory in Mathlib	14
9	Questions and Discussion	15
9.1	Can formalization reveal errors?	15
9.2	Has a major theorem such as the four-color theorem been formalized?	15
9.3	Why should one trust a proof assistant?	15
9.4	How closely does a formal proof follow the original paper proof?	15
9.5	Recommended learning resources	15
9.6	Editor choice	16
10	Closing Remarks	16
A	Compact Reference: Correspondence Used in the Talk	16
B	Minimal Standalone Lean-Style Skeleton	17

1 Opening and Overview

The seminar opened with a welcome to participants, including several visitors from physics, and introduced Jasper Robb from the University of Winnipeg. The announced topic was the Lean automated proving software system.

Lean is best understood as two closely related things:

- (1) a *proof assistant*, in which ordinary mathematical arguments can be translated into a formal language and checked by a computer; and
- (2) a *programming language*, equipped with types, functions, inductive definitions, and pattern matching.

The important point is that these are not separate layers. The proof-assistant behavior largely emerges from the programming-language foundations. The plan of the talk was therefore to learn the small computational core first, then reinterpret the same constructions as logic and proof.

Every Lean file automatically imports a foundational library called `Init`. It already provides Booleans, integers, lists, pairs, and many other standard objects. For pedagogical purposes, however, the talk rebuilt the essential types from scratch inside a namespace, so that the names would not conflict with the library versions.

2 Lean as a Programming Language

2.1 Namespaces and inductive types

A namespace prevents newly defined names from clashing with existing ones. The talk used an arbitrary namespace `H`:

```
namespace H

-- Definitions go here.

end H
```

The keyword `inductive` introduces a new type together with its *constructors*: the permitted ways of building elements of that type.

2.2 Booleans

A Boolean has exactly two constructors, corresponding to truth and falsity:

```
namespace H

inductive Bool : Type where
| true  : Bool
| false : Bool

#check Bool.true -- H.Bool
```

```
#check Bool.false -- H.Bool
#check Bool       -- Type

end H
```

The command `#check` asks Lean for the type of an expression. Thus `Bool.true : Bool`, while the type `Bool` itself has type `Type`.

2.2.1 Negation by pattern matching

A function can inspect which constructor was used to build its input. Boolean negation is therefore defined by two cases:

```
namespace H.Bool

open H

def not : H.Bool → H.Bool
| H.Bool.true  => H.Bool.false
| H.Bool.false => H.Bool.true

#check not
#eval not H.Bool.false

end H.Bool
```

The arrow `→` can be entered in a typical Lean editor using a backslash command such as `\r` followed by completion. Function application is written by juxtaposition: `not false`, rather than with mandatory parentheses.

2.2.2 Disjunction, NAND, and conjunction

A binary Boolean function has type `Bool → Bool → Bool`. The arrows associate to the right, so this means `Bool → (Bool → Bool)`: a function taking one Boolean and returning a function that takes the second Boolean.

```
namespace H.Bool

open H

def or : H.Bool → H.Bool → H.Bool
| H.Bool.false, H.Bool.false => H.Bool.false
| _, _ => H.Bool.true

#eval or H.Bool.false H.Bool.true

def nand : H.Bool → H.Bool → H.Bool :=
  fun a b => or (not a) (not b)

def and : H.Bool → H.Bool → H.Bool :=
  fun a b => not (nand a b)
```

```
#eval and H.Bool.true H.Bool.false
#eval and H.Bool.true H.Bool.true

end H.Bool
```

Parentheses matter in expressions such as `or (not a) (not b)`: without them, Lean would parse the expression as though the function `not` itself were being passed as an argument.

2.3 The unit and empty types

Two types simpler than the Booleans play a central logical role.

2.3.1 The unit type

The unit type has exactly one constructor and therefore exactly one canonical inhabitant:

```
namespace H

inductive Unit : Type where
  | unit : Unit

#check Unit.unit

end H
```

2.3.2 The empty type

The empty type has no constructors:

```
namespace H

inductive Empty : Type

end H
```

No closed term of type `Empty` can be constructed from this definition.

2.3.3 Functions between Unit and Empty

The possible function types illustrate an important asymmetry:

```
namespace H

def emptyToUnit : Empty → Unit :=
  fun _ => Unit.unit

def emptyToEmpty : Empty → Empty :=
```

```

fun e => e

def unitToUnit : Unit → Unit :=
  fun _ => Unit.unit

-- Impossible to complete without a placeholder:
def unitToEmpty : Unit → Empty :=
  fun _ => sorry

end H

```

The first two functions can never be called on a closed input, because no closed term of type `Empty` exists. Nevertheless, they are valid functions: if an impossible input were supplied, the function body would specify what to do. By contrast, a total function `Unit → Empty` cannot be defined constructively, because `Unit.unit` is an available input but there is no output of type `Empty`. The placeholder `sorry` suppresses the error while recording an unproved obligation.

2.4 Natural numbers

It is unnecessary to introduce one constructor for every natural number. Instead, natural numbers are generated recursively from zero and successor:

```

namespace H

inductive Nat : Type where
  | zero : Nat
  | succ : Nat → Nat

namespace Nat

#check zero
#check succ zero
#check succ (succ zero)

end Nat
end H

```

The intended interpretation is

$$0 := \text{zero}, \quad 1 := \text{succ zero}, \quad 2 := \text{succ (succ zero)},$$

and so on. This representation simultaneously encodes counting and repeated action.

2.4.1 Addition

Addition can recurse on its second argument:

```

namespace H.Nat

open H

```

```

def add : H.Nat → H.Nat → H.Nat
| a, H.Nat.zero    => a
| a, H.Nat.succ b => H.Nat.succ (add a b)

#eval add (H.Nat.succ H.Nat.zero)
         (H.Nat.succ (H.Nat.succ H.Nat.zero))

end H.Nat

```

The defining equations are

$$\text{add}(a, 0) = a, \quad \text{add}(a, Sb) = S(\text{add}(a, b)).$$

For example,

$$\begin{aligned}
 1 + 2 &= \text{add}(S0, S(S0)) \\
 &= S(\text{add}(S0, S0)) \\
 &= S(S(\text{add}(S0, 0))) \\
 &= S(S(S0)) = 3.
 \end{aligned}$$

The computation is nothing more than repeated pattern matching on the second input.

2.5 Products and sums

2.5.1 Product types

Unlike the preceding examples, a product type takes two types as parameters. An element of $\text{Prod } \alpha \ \beta$ stores both an α and a β :

```

namespace H

inductive Prod α( β : Type) : Type where
| intro : α → β → Prod α β

#check Prod Bool Nat
#check Prod.intro Bool.true Nat.zero

end H

```

2.5.2 Sum types

A sum type stores an element of one of two possible types:

```

namespace H

inductive Sum α( β : Type) : Type where
| inl : α → Sum α β
| inr : β → Sum α β

```

```

variable (bar1 : Sum Bool Nat := Sum.inl Bool.true)
variable (bar2 : Sum Bool Nat := Sum.inr Nat.zero)

end H

```

A function consuming a sum generally performs pattern matching to determine whether the value was built with `inl` or `inr`.

3 Lean as a Proof Assistant

Key idea

Propositions are types, and proofs are terms inhabiting those types.

This principle is often called the *Curry-Howard correspondence*. A proposition is represented by a type. To prove the proposition is to construct a term of that type. A type with an inhabitant behaves like a true proposition; an uninhabited type behaves like a false proposition.

The unit type is therefore a model of truth: it has the inhabitant `Unit.unit`. The empty type models falsity: it has no constructors and therefore no proof term.

3.1 Logical connectives as type constructors

The types already introduced reproduce the usual logical operations.

Logic	Type-theoretic representation	Meaning of an inhabitant
$\top$	unit type	a canonical proof of truth
$\perp$	empty type	no proof exists
$P \wedge Q$	product type $P \times Q$	a proof of P and a proof of Q
$P \vee Q$	sum type $P + Q$	a proof of one disjunct
$P \rightarrow Q$	function type $P \rightarrow Q$	transforms proofs of P into proofs of Q
$\neg P$	function type $P \rightarrow \perp$	converts a proof of P into contradiction

Thus, for example, `Prod Unit Empty` is uninhabited because its second component would require an inhabitant of `Empty`; `Sum Unit Empty` is inhabited via its left constructor. Likewise, `Empty → Unit` and `Empty → Empty` are inhabited function types, whereas `Unit → Empty` is not.

3.2 Reintroducing the logical names

The talk rebuilt the same structures in the universe `Prop`, using names that align with logic:


```

namespace H

inductive True : Prop where
| intro : True

inductive False : Prop

inductive And (P Q : Prop) : Prop where
| intro : P → Q → And P Q

inductive Or (P Q : Prop) : Prop where
| inl : P → Or P Q
| inr : Q → Or P Q

def Not (P : Prop) : Prop := P → False

end H

```

For the purposes of the talk, Prop may be viewed as a universe of proposition-types. The logical definitions are structurally the same as the earlier programming types.

4 Elementary Proofs by Construction and Pattern Matching

4.1 Projection from a conjunction

The proposition

$$P \wedge Q \longrightarrow P$$

is simply the left projection from a product:

```

namespace H

theorem And.left {P Q : Prop} : And P Q → P
| And.intro hP _ => hP

theorem And.right {P Q : Prop} : And P Q → Q
| And.intro _ hQ => hQ

end H

```

The pattern `And.intro hP hQ` exposes the two proofs stored in the conjunction. Returning `hP` proves the first projection. The talk noted the celebratory confetti displayed by the editor after a proof is accepted.

4.2 The first De Morgan implication

One direction of De Morgan's law is constructively valid:

$$\neg(P \vee Q) \longrightarrow (\neg P \wedge \neg Q).$$

The proof follows the ordinary natural-deduction argument exactly. Assume $h : \text{Not } (O \vee P \vee Q)$. To prove $\text{Not } P$, assume $hP : P$; then $O \vee \text{inl } hP : O \vee P \vee Q$, and applying h yields false. The proof for $\text{Not } Q$ is symmetric.

```
namespace H

theorem not_or_to_and_not {P Q : Prop} :
  Not (O ∨ P ∨ Q) → And (Not P) (Not Q) :=
  fun h =>
    And.intro
      (fun hP => h (O.inl hP))
      (fun hQ => h (O.inr hQ))

end H
```

The phrase “modus ponens” becomes ordinary function application: a proof of $P \rightarrow Q$ applied to a proof of P produces a proof of Q .

4.3 The converse De Morgan implication

The converse is also constructive:

$$(\neg P \wedge \neg Q) \rightarrow \neg(P \vee Q).$$

After assuming the conjunction and then a proof of $P \vee Q$, perform cases on the disjunction. In the left case, apply $\neg P$; in the right case, apply $\neg Q$.

```
namespace H

theorem and_not_to_not_or {P Q : Prop} :
  And (Not P) (Not Q) → Not (O ∨ P ∨ Q) :=
  fun h =>
    fun hpq =>
      match hpq with
      | O.inl hP => And.left h hP
      | O.inr hQ => And.right h hQ

end H
```

Here $\text{And.left } h$ has type $\text{Not } P$, so applying it to $hP : P$ yields False.

5 Constructive Logic and Classical Logic

Consider the other familiar De Morgan implication:

$$\neg(P \wedge Q) \rightarrow (\neg P \vee \neg Q).$$

In classical logic it is valid, but the constructive tools developed so far do not suffice. From the information that P and Q are not both true, one cannot in general constructively decide which of P or Q fails.

5.1 Law of excluded middle

The classical principle needed is the law of excluded middle:

$$P \vee \neg P.$$

It can be added as an axiom:

```
namespace H

axiom excludedMiddle (P : Prop) : Or P (Not P)

end H
```

An axiom is not proved; it is declared as an available term of the specified type.

5.2 Classical proof of the remaining De Morgan implication

With excluded middle, the proof splits into two cases.

- If P holds, prove $\neg Q$: assuming Q gives $P \wedge Q$, contradicting $\neg(P \wedge Q)$.
- If $\neg P$ holds, the left disjunct is already available.

```
namespace H

theorem not_and_to_or_not {P Q : Prop} :
  Not (And P Q) → Or (Not P) (Not Q) :=
  fun h =>
    match excludedMiddle P with
    | Or.inl hP =>
      Or.inr (fun hQ => h (And.intro hP hQ))
    | Or.inr hnP =>
      Or.inl hnP

end H
```

This example marks part of the boundary between constructive and classical reasoning. Constructive proofs must explicitly build the asserted object; classical axioms provide additional proof terms, such as a decision between P and $\neg P$ even when no computational procedure deciding P is known.

6 Equality as an Inductive Family

6.1 Definition

Equality is itself represented by a proposition-valued inductive family. It has one constructor, reflexivity:

```
namespace H

inductive Eq α{ : Type} : α → α → Prop where
```

```

| refl (a :  $\alpha$ ) : Eq a a

#check Eq Bool.false Bool.true
#check Eq.refl Bool.false

end H

```

The expression `Eq Bool.false Bool.true` is a well-formed proposition, but it has no constructor-generated proof. By contrast, `Eq.refl Bool.false` proves `Eq Bool.false Bool.false`.

6.2 Definitional equality and computation

A proof by reflexivity can also succeed when the two displayed expressions are not textually identical but reduce to the same term after unfolding definitions. For example, the previously defined addition computes $1 + 1$ to 2, so reflexivity proves the corresponding equality:

```

namespace H

open Nat

example :
  Eq (add (succ zero) (succ zero))
    (succ (succ zero)) :=
  Eq.refl (succ (succ zero))

end H

```

The talk joked that this reached $1 + 1 = 2$ in roughly 156 lines, substantially faster than the famous foundational development by Russell and Whitehead.

6.3 Symmetry

Suppose $h : \text{Eq } a \ b$. Pattern matching on h leaves only the reflexive case, in which b is replaced by a and the goal becomes $a = a$:

```

namespace H

theorem Eq.symm  $\alpha$  { : Type} {a b :  $\alpha$ } : Eq a b  $\rightarrow$  Eq b a
| Eq.refl a => Eq.refl a

end H

```

6.4 Transitivity

The same elimination principle proves transitivity:

```

namespace H

```

```

theorem Eq.trans α{ α : Type} {a b c : α} :
  Eq a b → Eq b c → Eq a c
| Eq.refl a, h => h

end H

```

Once the first equality is matched by reflexivity, b is identified with a , and the remaining hypothesis already has the desired type.

6.5 Congruence under functions

If $a = b$, then $f(a) = f(b)$:

```

namespace H

theorem congrArg α{ β : Type} (f : α → β) {a b : α} :
  Eq a b → Eq (f a) (f b)
| Eq.refl a => Eq.refl (f a)

end H

```

Again, the entire proof is the reflexive case after pattern matching.

7 Associativity of Natural-Number Addition

The final formal proof in the talk established

$$(a + b) + c = a + (b + c)$$

for the recursively defined natural numbers and addition. The proof is by recursion, equivalently induction, on c .

7.1 Base case

When $c = 0$, the goal is

$$(a + b) + 0 = a + (b + 0).$$

By the defining equation $x + 0 = x$, both sides reduce to $a + b$, so reflexivity closes the goal.

7.2 Successor case

Suppose $c = S(n)$. After unfolding addition, the target becomes

$$S((a + b) + n) = S(a + (b + n)).$$

The recursive call provides

$$(a + b) + n = a + (b + n).$$

Applying congruence to the successor function yields the desired successor equality.

```
namespace H.Nat

open H

theorem addAssoc (a b : H.Nat) :
  (c : H.Nat) →
  H.Eq (add (add a b) c)
    (add a (add b c))
| H.Nat.zero =>
  H.Eq.refl (add a b)
| H.Nat.succ n =>
  H.congrArg H.Nat.succ (addAssoc a b n)

end H.Nat
```

This proof displays the core mechanism of induction in dependent type theory: a recursively defined proof term follows the same constructors as the inductive data.

8 How Lean Is Used in Practice

The deliberately foundational style of the talk is not the usual day-to-day style of Lean development. In actual mathematical work:

- standard notation replaces most explicit constructor names;
- *tactics* give step-by-step instructions for constructing proof terms;
- the library `Mathlib` supplies a large body of definitions and theorems;
- editors show the current proof state, report type errors immediately, and provide interactive completion.

A modern proof may therefore look much closer to a structured mathematical derivation than to the low-level constructor code used above. Nevertheless, the resulting tactic script is elaborated into a proof term that is checked by the kernel.

8.1 Graph theory in Mathlib

The talk briefly displayed graph-theoretic material already present in `Mathlib`: definitions of simple graphs and more general graphs, chromatic number, trees, graph products, and basic theorems such as the two-colorability of even cycles. The point was not that all major graph theory has already been formalized, but that the library is substantial enough to support nontrivial work.

The speaker's own research concerns formalizing results in combinatorics on words.

9 Questions and Discussion

9.1 Can formalization reveal errors?

Yes. To formalize a proof, one must understand every dependency and every inference precisely. Statements that are dismissed in prose as “trivial” may conceal missing hypotheses or nontrivial subarguments. The speaker reported having found errors through this process, often because formalization forces a much deeper reading of the proof.

9.2 Has a major theorem such as the four-color theorem been formalized?

The four-color theorem has been formalized in a proof assistant, though not necessarily in Lean in the example discussed during the seminar. Such a result is convincing only relative to one’s trust in the assistant and its foundations.

9.3 Why should one trust a proof assistant?

Trust is concentrated in the *kernel*, the small component that checks proof terms. A smaller kernel has two advantages:

- (1) fewer lines of trusted code in which an implementation defect could occur;
- (2) a smaller object for independent inspection and verification.

No computer system is metaphysically infallible: hardware faults, software bugs, or an unsound implementation are conceivable. Moreover, every assistant rests on a foundational formal system. Lean is based on dependent type theory, specifically a calculus of constructions enriched by inductive types. One may still ask whether those foundations are accepted. The practical advantage is that the trusted base is sharply delimited.

The discussion also mentioned Gödel’s incompleteness phenomenon: sufficiently expressive consistent formal systems cannot internally certify every aspect of their own correctness.

9.4 How closely does a formal proof follow the original paper proof?

The speaker estimated that the answer is roughly split. Sometimes the paper proof can be followed closely, after expanding omitted details. In other cases, the original organization is awkward for the prover, or computationally inefficient, so the argument is reshaped. Formalization is therefore not merely transcription; it can involve substantial proof engineering.

9.5 Recommended learning resources

The primary recommendation was the *Natural Number Game*, which teaches elementary properties of natural numbers, addition, multiplication, and order in a gamified

sequence of formal proofs. The official Lean website also distinguishes resources aimed at programmers from those aimed at mathematicians.

When searching online, use the term “Lean 4” rather than merely “Lean”, which is too ambiguous.

9.6 Editor choice

The speaker personally used Neovim, partly because of its customizability and its usefulness for the incremental code presentation. For most new users, however, Visual Studio Code was recommended as the best-supported and most popular option. Emacs support is also available.

10 Closing Remarks

The seminar concluded with thanks to the speaker and an invitation to continue the discussion over lunch. The following week coincided with Canada Day, so no seminar was planned for that date; the organizers intended to resume with a later July session.

A Compact Reference: Correspondence Used in the Talk

Mathematical idea	Lean/type-theoretic object	Constructor or proof principle
True	one-point type	provide its unique inhabitant
False	empty type	no constructor exists
Conjunction	product / And $P \ Q$	construct both components; eliminate by pattern matching
Disjunction	sum / Or $P \ Q$	choose <code>inl</code> or <code>inr</code> ; eliminate by cases
Implication	function type	lambda abstraction and function application
Negation	$P \rightarrow \text{False}$	assume P and derive contradiction
Equality	inductive family <code>Eq a b</code>	reflexivity plus dependent pattern matching
Induction	recursion over constructors	one proof branch per constructor
Classical reasoning	additional axioms	e.g. excluded middle

B Minimal Standalone Lean-Style Skeleton

The following block collects the central definitions in one place. It is intended as a readable reconstruction of the code demonstrated in the talk; exact syntax may require small adjustments depending on namespace and Lean version.

```

namespace H

inductive Bool : Type where
  | true | false

inductive Unit : Type where
  | unit

inductive Empty : Type

inductive Nat : Type where
  | zero
  | succ : Nat → Nat

namespace Nat

def add : Nat → Nat → Nat
  | a, zero   => a
  | a, succ b => succ (add a b)

end Nat

inductive Prod α( β : Type) : Type where
  | intro : α → β → Prod α β

inductive Sum α( β : Type) : Type where
  | inl : α → Sum α β
  | inr : β → Sum α β

inductive True : Prop where
  | intro

inductive False : Prop

inductive And (P Q : Prop) : Prop where
  | intro : P → Q → And P Q

inductive Or (P Q : Prop) : Prop where
  | inl : P → Or P Q
  | inr : Q → Or P Q

def Not (P : Prop) : Prop := P → False

inductive Eq α{ : Type} : α → α → Prop where
  | refl (a : α) : Eq a a

end H

```